HR Purivity

Interview Preparation Guide

A beginner-friendly walkthrough of the project: what it is, how it is built, why each choice was made, and how to explain it with confidence.

Inside this guide
1. The project in plain words and the 30-second pitch
2. Key terms explained from zero
3. Architecture and why it looks this way
4. Tech stack and the reason for each tool
5. Step-by-step workflow (bullet points to memorize)
6. Query router, the brain of the app
7. The three iterations, the AI attempt, bugs and lessons
8. What I would improve
9. Interview questions with ready answers
10. Quick recall sheet

1. The Project in Plain Words

What is it?

- **HR Purivity** is a set of web-app prototypes built with **Streamlit** (a Python tool for making web pages).
- A user uploads **HR CSV files** (personal info, attendance, leave, performance).
- The user types a question in **everyday English**, such as *"show attendance by department"*.
- The app answers with **tables, charts and a downloadable report**.

What problem does it solve?

- HR data lives in **several separate CSV exports**.
- HR staff usually do **not know SQL or Python**.
- They still need answers like *"who are the low performers?"*
- **Solution:** a chat-style screen on top of the data that sends each question to the right pandas analysis.

The 30-second pitch

SAY THIS - OPENING PITCH

HR Purivity is a Streamlit-based HR analytics prototype. The user uploads HR CSV files, types a question in plain English, and gets back tables, charts, and a downloadable report. It went through three iterations: rule-based leave analysis, a generic CSV chatbot, and finally a multi-file HR analyzer. Technically it is a pandas pipeline wrapped in a Streamlit UI, with a keyword-based query router on top.

One-line version (if they want it very short)

SAY THIS - ONE LINER

It is a pandas analytics engine behind a Streamlit chat screen, so HR people can ask questions about their data without writing code.

2. Key Terms Explained From Zero

Term	Meaning in everyday words	Where it shows up here
Streamlit	Python library that turns a script into a web page. No HTML or JavaScript needed.	The whole user interface
pandas	Python library for working with tables of data (like Excel in code).	Reading, merging, grouping, counting
CSV	A plain text file where each line is a row and commas separate columns.	The HR data files
DataFrame	A pandas table held in memory.	The merged employee table (100 rows x 22 columns)
Merge / join	Combining two tables using a shared column.	Joining 4 files on employee_id
Outer join	Keep every row from both tables, even if one side has no match.	Keeps all employees
Normalization	Cleaning column names to one consistent style.	' Employee ID ' becomes employee_id
Router	Code that decides which function should handle a request.	Keyword if/elif chain
Regex	A pattern used to find text, such as an ID inside a sentence.	Pulling the employee ID out of a question
Markdown	Light text format for tables and headings.	How answers and the report are formatted
LLM	Large language model (an AI that understands text), such as Groq or Ollama models.	Tried in app4.py and app7.py
Grounding	Giving the AI the real data or schema so its answer is based on facts.	The key lesson of the AI attempt
Text-to-SQL	AI turns a question into a database query, then the query runs.	What I would build next
Cache	Saving a result so it is not recomputed every time.	Missing here; I would add st.cache_data
Session state	Memory that survives between Streamlit reruns.	Missing here; I would add st.session_state
Schema drift	Data columns differ from what the code expects.	Caused the KeyError on 'name'

3. Architecture

The big picture (top to bottom)

Architecture points to memorize

- It is a **single-process, local, single-user** app. No backend server, no database, no login, no tests.
- Everything runs **top to bottom in one Python script**. Streamlit reruns it on every click.
- There is **no AI in the main path**. The intelligence is a deterministic keyword router.
- Data flow: **files -> pandas DataFrame -> router -> analysis function -> Markdown/chart -> screen**.
- Each analysis follows one pattern: **Guard, Compute, Format, Return**.

Why this architecture?

- **Fast to build:** a prototype to prove the idea needed a working screen in hours, not weeks.
- **No setup for the user:** runs locally with one command, no server or database to install.
- **Predictable:** the same question always gives the same answer, which is easy to debug.
- **Right tool for tabular data:** pandas already does grouping, counting and averaging.
- **Honest trade-off:** it does not scale, is not secure for many users, and the router is brittle.

SAY THIS - ARCHITECTURE ANSWER

I chose a single-script Streamlit plus pandas design because the goal was to validate the idea quickly with non-technical users. It has no backend or database, which keeps it easy to run, and the trade-off is that it is a local single-user prototype. For production I would move to a text-to-SQL backend with caching and authentication.

4. Tech Stack and Why Each Tool

Layer	Technology	Version	Why it was chosen
Language	Python	3.12.8	Best ecosystem for data work
UI	Streamlit	1.29.0 (P1), 1.41.1 (P2, P3)	Web UI in pure Python, no frontend skills needed
Data	pandas	2.2.1 / 2.2.3	Group, merge, filter and count tables
Numerics	NumPy	1.26.3 / 2.2.1	Numeric support behind pandas
Charts	matplotlib, seaborn	3.10.x / 0.13.2 (unpinned in P1)	Static charts shown with st.pyplot
Interactive charts	Plotly	5.24.1 / 6.0.1 (P2)	Hover and zoom charts; unused in P3
Excel	openpyxl	3.1.5	Lets pandas read .xlsx files
Tables	tabulate	0.9.0	Required by to_markdown()
Regex	re (standard library)	Python 3.12	Extract IDs and dates from questions
Regex	regex package	2023.12.25	Declared but never imported
LLM (P1)	groq SDK	0.20.0 (unpinned)	Tried hosted AI in app4.py
LLM (P3)	ollama, torch, transformers	0.4.6 / 2.5.1 / 4.48.0	Tried local AI models
PDF	reportlab	4.2.5 (P2)	PDF export
PNG export	kaleido	absent	Needed by fig.to_image, never installed
ML	scikit-learn	used in P2 app3.py	LabelEncoder defined, never called
Packaging	PyInstaller	dist/run_app.exe (P2)	Turn the app into an executable
Environment	venv	all three broken	Point to a missing Python install

Libraries actually imported, by file

File	Imports	Unused
Employee Nanalyser/app.py	streamlit, pandas, numpy, re	numpy
Employee Nanalyser/app1.py	streamlit, pandas, numpy, re, datetime, matplotlib.pyplot, seaborn	numpy
Employee Nanalyser/app3.py	streamlit, pandas, re	none
Employee Nanalyser/app4.py	streamlit, pandas, re, groq, os	re, os
Employee Nanalyser/app5.py	streamlit, pandas, re, groq	groq
Chat CSV/app.5.py	streamlit, pandas, numpy, plotly.express, plotly.graph_objs, reportlab, io, base64	matplotlib.pyplot, seaborn, reportlab.Image
Chat CSV/app3.py	streamlit, pandas, matplotlib.pyplot, seaborn, sklearn.LabelEncoder, difflib.get_close_matches	LabelEncoder (never called)
data vis/app.5.py	streamlit, pandas, numpy, re	numpy
data vis/app6.py	streamlit, pandas, numpy, plotly.express, plotly.graph_objs	numpy, plotly
data vis/app7.py	streamlit, pandas, numpy, typing, ollama, torch, transformers	numpy, torch, typing
data vis/app7.1.py	streamlit, pandas, numpy, ollama, json, re	numpy, json
data vis/app7.2.py	streamlit, pandas, numpy, re	numpy

Why Streamlit and pandas, not something else?

- **Streamlit over Flask or React:** no HTML, CSS or JavaScript to write. The goal was data answers, not web design.
- **pandas over a database:** the data arrives as CSV files, so loading straight into memory avoids a setup step.
- **Keyword router over an LLM:** free, fast, predictable, and works offline.
- **matplotlib and seaborn:** reliable, familiar, and they work with st.pyplot.

5. Step-by-Step Workflow (Memorize These Bullets)

How to use this section: each step has three parts. **What** happens, **How** it is done, and **Why** it was done that way. If an interviewer asks *"how did you do this?"* give the What and How. If they ask *"why?"* give the Why.

STEP 0: Opening pitch

What happens

- Introduce the project in one breath.

How it is done

- Streamlit app, CSV upload, plain English question, tables/charts/report.
- Mention three iterations and the keyword router.

Why this way

- Gives the interviewer a map before the details.

SAY THIS

HR Purivity is a Streamlit-based HR analytics prototype. The user uploads HR CSV files, types a question in plain English, and gets back tables, charts, and a downloadable report. Technically it is a pandas pipeline wrapped in a Streamlit UI, with a keyword-based query router on top.

STEP 1: UI layer and startup

What happens

- The app starts with streamlit run app1.py.
- The page title, icon and wide layout are set.
- A title, an expander with example questions, and a file uploader appear.

How it is done

- First call is st.set_page_config().
- Uploader uses accept_multiple_files=True and accepts CSV, XLSX, XLS.
- If no file is uploaded, an info message shows and the script stops.
- Everything is top-to-bottom Python. Streamlit reruns the whole script on every interaction.

Why this way

- No separate frontend means less code to maintain.
- Example questions teach non-technical users how to ask.

SAY THIS

The app starts with streamlit run app1.py. The first call is st.set_page_config, which sets the title, icon and wide layout. Then it shows the title, an expander with example questions, and a multi-file uploader for CSV, XLSX and XLS. If nothing is uploaded it shows an info message and stops. Everything is top-to-bottom Python, and Streamlit reruns the whole script on every interaction.

STEP 2: File ingestion

What happens

- Uploaded files are turned into pandas DataFrames.
- An analyzer class, AdvancedHRDataAnalyzer, is created.

How it is done

- Each file: try pd.read_csv() first.
- If that fails, fall back to pd.read_excel().
- Success shows st.success. Failure is caught and shown as st.error.
- This happens again on every rerun because there is no caching.

Why this way

- Users export from different tools, so both formats must work.
- Per-file error handling means one bad file does not crash the app.

SAY THIS

For each uploaded file the app tries read_csv first and falls back to read_excel. Success shows a green message, and failures are caught and shown as an error, so one bad file does not crash the app. This repeats on every rerun because there is no caching.

STEP 3: Header normalization

What happens

- Column names are cleaned into one consistent style.

How it is done

- One pandas chain: .str.strip().str.lower().str.replace(' ', '_').
- Example: ' Employee ID ' becomes employee_id.
- Earlier drafts lowercased but did not strip, so trailing spaces broke every check.

Why this way

- All later code checks exact snake_case names.
- If normalization is skipped, every column check fails.

SAY THIS

After parsing I normalize column names with strip, lower and replace spaces with underscores. So ' Employee ID ' becomes employee_id. This matters because all downstream logic checks for exact snake_case names.

STEP 4: Multi-file merge (core data engineering)

What happens

- Four datasets become one table: personal info, attendance, leave, performance.
- With the bundled data the result is 100 rows by 22 columns.

How it is done

- merge_data() does an outer join on employee_id across all files.
- Duplicate non-key columns get a _drop suffix and are then dropped.
- If a file has no employee_id, it falls back to pd.concat(axis=1), aligning by row index.

Why this way

- employee_id exists in all four files, so it is the natural key.
- Outer join keeps every employee, even one missing from a file.
- Honest limitation: the _drop logic silently discards right-hand values, and concat can mix unrelated employees if row order differs.

SAY THIS

The four HR datasets share a common key, employee_id. merge_data does an outer join on that key across all files. Duplicate columns get a suffix and are dropped. If a file has no employee_id, it falls back to concat by row index. The result is a single 100 by 22 table. One honest caveat: the drop logic silently discards right-side values, and the concat fallback can mix employees if row order differs.

STEP 5: Query capture

What happens

- The user types a question.

How it is done

- st.text_input collects the text.
- The query is lowercased and stripped before routing.
- An Analyze button exists but does nothing in this version. Any non-empty query triggers analysis.

Why this way

- Lowercasing makes keyword matching case-insensitive.

STEP 6: Query routing (the brain)

What happens

- The app decides which analysis the question needs.

How it is done

- intelligent_query_handler() is an ordered if/elif chain over substrings.
- Order: employee details regex, performance, attendance, leave, department, salary, demographics, summary, general fallback.
- Employee lookup uses a regex to pull the ID from the text.
- Each branch checks a list of synonyms.
- If nothing matches, it falls back to matching words against column names.

Why this way

- Deterministic, fast, free, and runs offline.
- Trade-off: brittle, it only understands phrasing it was written for.
- First match wins, so order matters. "department salaries" is caught by department, not salary.

Route table

Intent	Keywords	Method
Employee lookup	employee, staff, worker, id	get_comprehensive_employee_details
Performance	performance, rating, review, appraisal	analyze_performance
Attendance	attendance, present, absent	analyze_attendance
Leave	leave, vacation, holiday, time off	analyze_leaves
Department	department, team, division, unit	analyze_departments
Salary	salary, pay, compensation, income	analyze_salaries
Demographics	demographic, age, gender, diversity	analyze_demographics
Report	summary, overview, report, analysis	generate_comprehensive_report
Fallback	none of the above	general_data_analysis

SAY THIS

The intelligence is a keyword and regex router, not an LLM. It is an ordered if/elif chain: employee details, performance, attendance, leave, department, salary, demographics, summary, then a general fallback. It is deterministic and fast, but brittle, because it only understands phrasing it was written for.

STEP 7: Analysis execution

What happens

- The chosen analysis function runs on the merged table.
- Six domains: employee lookup, performance, attendance, leave, departments, salary plus demographics.

How it is done

- Pattern for every method: **Guard, Compute, Format, Return**.
- Guard: check required columns exist, else return "No data found".
- Compute: pandas groupby, value_counts, mean, min, max.

- Format: build a Markdown table.
- Return: a Markdown string shown with st.markdown().

Why this way

- One repeated pattern makes the code easy to read and extend.
- Guards prevent crashes on missing columns.

SAY THIS

Every analysis method follows the same pattern. First a guard to check the required columns exist, then pandas computation with groupby, value_counts and averages, then formatting as a Markdown table, and finally returning the string for st.markdown.

STEP 8: Chart rendering

What happens

- Visuals appear next to the tables.

How it is done

- Charts are built with matplotlib and seaborn and shown with st.pyplot().
- Each chart is drawn as a side effect inside the analysis method.

Why this way

- These libraries are reliable and well known.
- Design weakness: analysis and presentation are coupled, which makes testing harder.
- Better design: return the figure object and let the UI decide how to show it.

SAY THIS

Charts use matplotlib and seaborn, rendered through st.pyplot. They are drawn inside the analysis methods as a side effect. That is a design weakness because analysis and presentation are coupled. A cleaner design returns the figure and lets the UI render it.

STEP 9: Report generation and download (the key bug)

What happens

- The user can download one combined report.

How it is done

- generate_comprehensive_report() concatenates all six analyses into one Markdown string with a timestamp.
- It is passed to st.download_button() as hr_analytics_report.md.
- The button calls the report function on every rerun, so every analysis re-runs.
- The performance table selects a name column. The data only has first_name and last_name, so a **KeyError: 'name'** is raised on every query.

Why this way

- Shows two real problems: schema drift (code expects a column the data lacks) and unintended re-execution (work repeated on each rerun).
- Result: app1.py is unusable with the bundled data.

SAY THIS

The report function concatenates all six analyses into one Markdown file for download. The critical bug is that the download button re-runs every analysis on each rerun, including a performance table that selects a name column. The data only has first_name and last_name, so it raises KeyError name on every query. It is a good example of schema drift and unintended re-execution.

STEP 10: Streamlit rerun model

What happens

- Understanding why the app feels slow and forgetful.

How it is done

- Streamlit reruns the whole script on every upload and click.
- The analyzer is rebuilt each time, so files are re-parsed and re-merged.
- No st.session_state and no st.cache_data.

Why this way

- Fix: @st.cache_data on the parsing function, st.session_state for the analyzer instance.

SAY THIS

Streamlit reruns the entire script on every interaction, so the analyzer is rebuilt and the files are re-parsed and re-merged each time. There is no session state and no caching. The fix is st.cache_data on parsing and st.session_state for the analyzer.

6. Module Map (File by File)

Project 1: Employee Nanalyser

File	Lines	Role	Key functions
app.py	132	Minimal keyword analyzer, per-file lookups, 5 intents, no merge	process_uploaded_files, intelligent_query_handler, analyze_performance, analyze_attendance, analyze_leaves
app1.py	**635**	**Best version:** CSV/Excel ingest, merge, 6 domains, charts, report	merge_data, analyze_performance, analyze_attendance, analyze_leaves, analyze_departments, analyze_salaries, analyze_demographics, generate_comprehensive_report
app3.py	120	Rule-based analyzer, renders directly to Streamlit	process_uploaded_files, intelligent_query_handler, analyze_attendance, analyze_leaves, analyze_departments
app4.py	77	Groq passthrough: parses CSVs but sends only the question	intelligent_query_handler
app5.py	125	Near-identical to app3.py plus an unused Groq client	intelligent_query_handler, analyze_attendance, analyze_leaves, analyze_departments

Project 2: Chat CSV Final - Copy

File	Lines	Role	Key functions
app.5.py	995	Monolith: upload, chat, Plotly charts, HTML/PNG/PDF export	AdvancedDataAnalyzer, comprehensive_analysis, _determine_visualization_type, 4 chart builders, generate_professional_report (defined twice)
app3.py	115	**Only working file:** prompt-to-chart generator with code display	generate_graph, generate_python_code, encode_categorical_columns
run_app.py	3	Launcher: os.system("streamlit run app.py")	none
run_app.spec	38	PyInstaller build spec	config only

Project 3: data visulaization

File	Lines	Role	Key functions
app.5.py	166	Single-CSV leave analyzer, regex date and department filters	prepare_data, _extract_date_range, _extract_department, _generate_intelligent_response
app6.py	128	Single-CSV group-by keyword bot	intelligent_query_handler, _generate_leave_insights
app7.py	242	Multi-CSV plus Ollama/Hugging Face (context discarded)	_initialize_language_model, _generate_ollama_response, _generate_huggingface_response
app7.1.py	250	Multi-CSV keyword plus broken ollama.query fallback	_get_comprehensive_employee_details, _gender_count_analysis, _ollama_advanced_analysis
app7.2.py	**132**	**Safest runnable:** multi-CSV keyword over exact snake_case columns	process_uploaded_files, get_comprehensive_employee_details, analyze_performance, analyze_attendance, analyze_leaves, analyze_departments

7. Three Iterations, the AI Attempt, Bugs and Lessons

The three projects (iterations, not three products)

#	Folder	What it explores	Status
1	Employee Nanalyser	HR-specific: 4 linked CSVs, merge on employee_id, 6 domains, charts, Markdown report	Best version; app1.py has the KeyError 'name' bug
2	Chat CSV Final - Copy	Generic CSV chatbot plus Plotly plus PDF export	Most broken; main app.py missing, app.5.py does not compile
3	data visulaization	Earliest drafts: single-CSV leave bots, multi-CSV LLM attempts	Broken environment; zero charts despite the name

- Same skeleton in all three: **Streamlit -> uploader -> pandas -> keyword router -> Markdown or chart output**.
- No shared code, no database, no authentication, no tests, no backend. All are local, single-user prototypes.

SAY THIS

The three folders are three iterations of the same idea, not three products. Each one explored a different angle: leave analysis, a generic CSV chatbot, and a multi-file HR analyzer. Looking back, they are a symptom of copy-paste iteration without version control.

Problem to approach mapping

Problem	Approach taken	Result
Non-technical users cannot query CSVs	Chat/text-input interface in Streamlit	Works, but routing is brittle
Data spread across 4 files	Outer join on employee_id	Works in app1.py only
Want charts	matplotlib/seaborn via st.pyplot	Works in app1.py only
Want downloadable report	Concatenate all analyses, Markdown download	Crashes due to the name column bug
Want AI answers	Groq SDK in app4.py	Failed: data never sent to the model
Want multi-file analysis	Multi-upload in app7.py / app7.1.py	Works for parsing, broken for LLM
Want natural language understanding	Keyword/regex if/elif routing	Deterministic but brittle

The AI (LLM) attempt

SAY THIS

I wired in the Groq SDK in one draft, app4.py. But I made a critical mistake: I sent only the user's question to the model, never the CSV data or schema, so the model could not answer data questions. The lesson is that grounding matters more than the model. You must put the schema and sample rows in the prompt, or use text-to-SQL. That is the biggest technical takeaway from this project.

- Groq SDK in app4.py (Project 1).
- app5.py builds a client but never uses it.
- data visulaization/app7.py has the same flaw: context is built, then discarded (line 89).
- app7.1.py calls ollama.query, which does not exist, giving an AttributeError.
- **No CSV content ever reached an LLM.**

Key bugs and lessons

Bug	Location	Lesson
KeyError: 'name'	app1.py:190	Schema drift: validate column contracts
LLM never got the data	app4.py:42, app7.py:89-104	Grounding matters more than the model
ollama.query does not exist	app7.1.py:233	Verify library APIs before calling
Indentation error	app.5.py:355 (Chat CSV)	13 duplicated UI blocks caused it
Missing app.py	run_app.py:3	Launcher had no target
No caching	All apps	Files re-read on every rerun
Hardcoded schemas	All apps	Missing columns crash silently or loudly
Plaintext Groq key	secrets.toml	Never commit secrets
Broken .venv	All three	Environment portability
Data quality issues	datasets/	99 of 100 attendance rows violate totals

8. What I Would Improve

- **1. Replace keyword routing** with text-to-SQL, or an LLM that receives the schema and sample rows in the prompt.
- **2. Validate column contracts at upload time** and tell the user exactly what is missing.

- **3. Add st.cache_data and st.session_state** so files are not re-parsed on every click.
- **4. Separate analysis from presentation:** return DataFrames and figures, let the UI render them.
- **5. Use version control from day one.** The three folders are a symptom of copy-paste iteration.

SAY THIS - IMPROVEMENTS

If I rebuilt this today, I would make five changes. First, replace keyword routing with text-to-SQL or an LLM with the schema and sample rows in the prompt. Second, validate column contracts at upload time. Third, add st.cache_data and st.session_state. Fourth, separate analysis from presentation. And fifth, use version control from day one.

How I would scale it

- Text-to-SQL backend on a real database.
- Caching and session state.
- Authentication and role-based access (HR data is sensitive).
- Containerize with Docker for repeatable deployment.
- Move secrets to environment variables or a secrets manager.

9. Interview Questions With Ready Answers

Answer style: give the **direct answer first** in one sentence, then **one reason**, then **one honest trade-off**. Interviewers trust candidates who name limitations.

Q1. What does your project do?

A: It is a Streamlit HR analytics app. Users upload CSV files, ask questions in plain English, and get tables, charts and a downloadable report.

Q2. What is the architecture?

A: A single-script, layered pipeline: Streamlit UI, ingestion, header cleaning, multi-file merge on employee_id, a keyword router, pandas analysis methods, and a Markdown/chart output layer. It is local and single-user with no backend or database.

Q3. Explain it step by step.

A: Upload files. Read each with read_csv, falling back to read_excel. Normalize headers to snake_case. Outer-join all files on employee_id. Capture the question. Route it by keywords and regex. Run the matching pandas analysis (guard, compute, format). Show the Markdown table and chart. Offer a combined report for download.

Q4. How does it understand the user's question?

A: Through a keyword and regex router, not an LLM. An ordered if/elif chain checks synonym lists for each intent, and a regex extracts employee IDs.

Q5. Why did you choose a keyword router instead of an LLM?

A: It is free, fast, predictable and works offline, which suited a prototype. The trade-off is brittleness: it only handles phrasing it was written for.

Q6. Why Streamlit?

A: It builds a web interface in pure Python, so I could focus on the data logic instead of frontend code.

Q7. Why pandas and not a database?

A: The data arrives as CSV files, so loading it into memory avoids setup. For production I would move to a database with text-to-SQL.

Q8. Why an outer join?

A: I wanted to keep every employee even if they are missing from one file, so no one disappears from the analysis.

Q9. What is the merge key and why?

A: employee_id, because it appears in all four files and uniquely identifies a person.

Q10. What happens if a file has no employee_id?

A: It falls back to concat along columns, aligning by row index. That is risky because unrelated employees can be mixed if row order differs.

Q11. Why normalize column names?

A: All downstream checks use exact snake_case names. Without normalization a single trailing space breaks every column check.

Q12. Did you use AI?

A: Yes, I tried Groq and Ollama, but I sent only the question and never the data, so the model could not answer. The lesson: grounding matters more than the model.

Q13. How would you fix the AI part?

A: Put the schema and sample rows in the prompt, or use text-to-SQL so the model writes a query that actually runs on the data.

Q14. What was the biggest bug?

A: KeyError: 'name' in app1.py. The code selected a name column but the data only had first_name and last_name. It is schema drift, made worse by the download button re-running every analysis on each rerun.

Q15. How did you handle errors?

A: Per-file try/except on upload, and a guard at the start of each analysis method that checks required columns exist.

Q16. What is the weakness in your design?

A: Analysis and presentation are coupled: charts are drawn as side effects inside analysis methods. I would return data and figures and let the UI render them.

Q17. Why does the app re-read files on every click?

A: Streamlit reruns the whole script on each interaction and I did not use caching. The fix is st.cache_data and st.session_state.

Q18. How would you scale it?

A: Text-to-SQL backend, caching, authentication, and Docker containerization.

Q19. How would you secure it?

A: Add login and role-based access, keep secrets out of the repo (the Groq key was in plaintext), and encrypt or restrict sensitive HR data.

Q20. How would you test it?

A: Separate analysis functions from the UI so they return DataFrames, then write unit tests with small sample CSVs, plus a schema validation test for required columns.

Q21. Why are there three folders?

A: They are three iterations of the same prototype, not three products. It showed me I should use version control from the start.

Q22. What did you learn?

A: Grounding matters more than the model, validate schemas early, cache expensive work, separate logic from UI, and use version control.

Q23. Which version works best?

A: Employee Nanalyser/app1.py is the most complete, but it has the name bug. app7.2.py in Project 3 is the safest runnable one.

Q24. What are the limitations?

A: Brittle routing, no real language understanding, no AI grounding, hardcoded schemas, no caching, local single-user only, and data quality problems in the sample datasets.

Q25. What would you do differently?

A: Start with a schema contract, use version control, ground the LLM with schema and sample rows, and build analysis as testable functions.

10. Quick Recall Sheet

#	Phase	One word	One line
0	Pitch	Overview	Streamlit HR analytics: upload, ask, tables/charts/report
1	UI	Startup	set_page_config, title, uploader
2	Ingest	Files	read_csv else read_excel, per file, every rerun
3	Normalize	Headers	strip, lower, replace to snake_case
4	Merge	Join	outer join on employee_id, 100x22 frame
5	Query	Input	text input, lowercased
6	Route	Brain	if/elif over keywords plus regex for ID
7	Analyze	Compute	guard columns, pandas, Markdown
8	Chart	Visuals	matplotlib/seaborn via st.pyplot
9	Report	Export	concatenate all, download, KeyError bug
10	Rerun	Streamlit	no cache, rebuilt on every interaction

Closing line (memorize)

SAY THIS - CLOSING

In one line: it is a pandas analytics engine behind a Streamlit chat UI, with a keyword router as the brain, a multi-file merge as the core data idea, and a set of honest limitations (brittle routing, no LLM grounding, and schema-drift bugs) that taught me what a production version needs: real text-to-SQL, schema validation, and caching.

Last-minute checklist

- Say the **opening pitch** out loud.
- Know the **7 architecture layers** in order.
- Know the **route table** and that first match wins.
- Remember the **KeyError: 'name'** bug and why it happened.

- Remember the **grounding lesson** from the AI attempt.
- Finish with the **five improvements**.